## Documentation – Application E-commerce "Stubborn"

### Présentation du projet

L'application **Stubborn** est un site e-commerce permettant aux utilisateurs de consulter des sweat-shirts, les ajouter à un panier et effectuer un achat en ligne.

Le projet a été développé avec le framework **Symfony** en respectant une architecture MVC.

---

### Technologies utilisées

- PHP 8.5
- Symfony
- Doctrine ORM
- MySQL
- Twig
- Stripe (paiement)
- HTML / CSS /javascript

---

### Authentification

#### Inscription

- L'utilisateur peut créer un compte avec :
  - Nom
  - Email
  - Mot de passe
  - Adresse
- Un email de confirmation est envoyé automatiquement.
- Le compte est activé uniquement après validation du lien reçu par email.

#### Connexion

- L'utilisateur peut se connecter avec :
  - Son **nom OU son email**
  - Son mot de passe

- Vérifications :
    - L’utilisateur doit exister
    - Le compte doit être vérifié
- Gestion via **Symfony Security**

---

**Gestion des produits**

**Fonctionnalités**

- Affichage des sweat-shirts :
    - Image
    - Nom
    - Prix
- Page détail produit :
    - Choix de la taille
    - Bouton "Ajouter au panier"

**Back-office (Admin)**

- Ajout d’un produit
- Modification
- Suppression

---

**Panier**

**Fonctionnement**

- Chaque utilisateur possède un panier en base de données
- Les produits sont stockés avec :
    - Quantité
    - Taille

**Actions possibles**

- Ajouter un produit
- Augmenter la quantité

- Diminuer la quantité
- Supprimer un produit

**Calcul**

- Total calculé dynamiquement :

prix × quantité

---

**Paiement avec Stripe**

**Mise en place**

Un **service Stripe** a été créé pour gérer les paiements.

**Service utilisé**

```
class StripeService
{
   public function createPaymentIntent(int $amount): PaymentIntent
   {
      Stripe::setApiKey($this->secretKey);


      return PaymentIntent::create([
         'amount' => $amount,
         'currency' => 'eur',
      ]);
   }
}
```

**Utilisation dans le contrôleur**

```
$paymentIntent = $this->stripeService->createPaymentIntent($amount);
```

**Fonctionnement**

1. Calcul du total du panier
2. Création d'un PaymentIntent
3. Envoi au frontend (client_secret)(twig ici)

4. Paiement en mode test

**Mode test**

- Utilisation des clés Stripe :
  - STRIPE_SECRET_KEY
  - STRIPE_PUBLIC_KEY

---

🧪 **Tests**

Des tests ont été réalisés pour :

- Ajout au panier
- Modification des quantités (ajouter ou réduire)
- Supprimer
- Si produit existe
- Tester l'accès à la  route checkout (paiement)
- login

commande pour lancer

- php bin/phpunit

Résultats visibles dans :

- l'interface
- la console

---

**Accès à l'application**

L'application est accessible en local via :

http://localhost:8000

Pré-requis :

- WAMP lancé
- Base de données active

---

**Base de données**

**Tables principales**

- user
- sweatshirt
- cart
- cart_item

**Relations**

- Un utilisateur → plusieurs paniers
- Un panier → plusieurs produits

---

**Fonctionnalités principales**

✔ Inscription avec email de vérification
✔ Connexion sécurisée
✔ Catalogue produits
✔ Panier dynamique
✔ Paiement Stripe (test)
✔ Back-office admin

---

**Problèmes rencontrés**

- Configuration PHP (extensions manquantes : PDO, cURL)
- Gestion de l'authentification (email vs username)
- Intégration Stripe

**Solutions**

- Activation des extensions dans php.ini
- Création d'un authenticator personnalisé
- Création d'un service Stripe

---

- **Comptes de test admin**

nom : admin

email: smartbrief.me@gmail.com

password: admin1234

## ✅ Conclusion

Le projet répond aux exigences demandées :

- Application fonctionnelle
- Authentification complète
- Gestion des produits et du panier
- Paiement en mode test avec Stripe
- Architecture propre avec services

---

## 👨‍💻 Auteur

Projet réalisé par :
**Desteur Piepenche Fall**

**Année : 2026**